

AI 보험금 산정 및 손해사정 지원 시스템 ERD 설계서

AI 보험금 산정 및 손해사정 지원 시스템 ERD 설계서

문서번호: DB-ERD-001

버전: v1.0

기준일: 2026-08-18

상위문서: AI 보험금 산정 및 손해사정 지원 시스템 요구사항정의서 v1.0

연계문서:

- ARCH-FUNC-001 시스템 기능구조도
- UX-FLOW-001 사용자 흐름도
- UI-SPEC-001 화면설계서

MVP 범위: 정액형 제3보험 보험금 예상 산정 지원

I . ERD 설계 목적

본 ERD는 다음 전체 흐름을 데이터베이스 구조로 표현한다.

```
graph TD
    A[사용자] --> B[피보험자]
    B --> C[보험계약]
    C --> D[가입담보]
    D --> E[보험사고 Claim]
    E --> F[의료·사고문서]
    F --> G[OCR]
    G --> H[검증된 Fact]
    H --> I[약관 Version]
    I --> J[담보 Rule]
```

↓
지급조건 판단
↓
보험금 계산
↓
근거 Evidence
↓
전문가 Review
↓
Audit

II. 핵심 설계 원칙

1. 보험회사 상품정보와 사용자 계약정보를 분리한다

InsuranceProduct
≠
InsuranceContract

InsuranceProduct 는 보험회사가 판매하는 표준상품이고,

InsuranceContract 는 특정 사용자가 실제 가입한 계약이다.

2. 표준담보와 계약담보를 분리한다

Coverage
≠
ContractCoverage

예:

Coverage
암진단비

↓

ContractCoverage
김○○ 가입계약
암진단비
가입금액 30,000,000원

3. 약관과 약관 Version을 분리한다

```
Policy
↓
PolicyVersion
↓
PolicyClause
```

약관 개정 시 기존 Version을 수정하지 않고 새로운 Version을 생성한다.

4. Rule 역시 Version을 관리한다

```
BenefitRule
↓
RuleVersion
```

과거 보험금 계산은 계산 당시 Rule Version을 계속 참조해야 한다.

5. OCR 원본과 사용자가 검증한 Fact를 분리한다

```
OCRResult
↓
ExtractedFact
↓
VerifiedFact
```

AI/OCR이 읽은 값을 보험금 계산에 곧바로 사용하지 않는다.

6. 보험금 계산결과는 Snapshot을 보관한다

계약이나 Rule이 나중에 변경되어도 과거 결과가 바뀌어서는 안 된다.

따라서 계산 시 다음을 Snapshot으로 저장한다.

- 가입금액
- 지급률
- Rule Version
- Policy Version

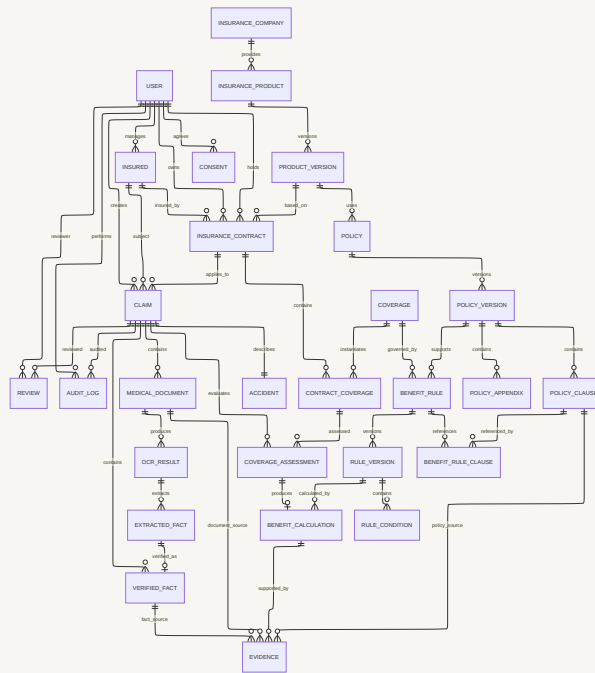
- 입력 Fact
 - 계산식
 - 계산결과
-

III. 전체 Domain

데이터베이스를 다음 9개 Domain으로 구분한다.

Domain	주요 Entity
사용자	User, Insured, Consent
보험상품	InsuranceCompany, InsuranceProduct, ProductVersion
계약	InsuranceContract, ContractCoverage
약관	Policy, PolicyVersion, PolicyClause, PolicyAppendix
Rule	Coverage, BenefitRule, RuleVersion, RuleCondition
Claim	Claim, Accident
문서·AI	MedicalDocument, OCRResult, ExtractedFact, VerifiedFact
판단·계산	CoverageAssessment, BenefitCalculation, Evidence
운영	Review, AuditLog

IV. 전체 논리 ERD



V. 핵심 관계 요약

```

InsuranceCompany
  1
  |
  N
InsuranceProduct
  1
  |
  N
ProductVersion

```

```

Policy
  1
  |
  N
PolicyVersion
  1
  |
  N
PolicyClause

```

```

User
  1

```

```

|
N
InsuranceContract
1
|
N
ContractCoverage

```

```

Claim
1
|
N
MedicalDocument
1
|
N
OCRResult
1
|
N
ExtractedFact

```

```

Claim
1
|
N
CoverageAssessment
1
|
0..1
BenefitCalculation

```

VI. 사용자 Domain

1. USER

사용자 계정 테이블.

컬럼	타입	Key	설명
user_id	UUID	PK	사용자 ID
email	VARCHAR(255)	UK	로그인 이메일
password_hash	VARCHAR(255)		비밀번호 Hash

컬럼	타입	Key	설명
display_name	VARCHAR(100)		이름/표시명
phone	VARCHAR(30)		전화번호
role	VARCHAR(30)		USER/ADJUSTER/ADMIN 등
status	VARCHAR(20)		ACTIVE/LOCKED/DELETED
mfa_enabled	BOOLEAN		MFA 사용 여부
created_at	TIMESTAMP		생성일
updated_at	TIMESTAMP		수정일
deleted_at	TIMESTAMP		Soft Delete

주요 Index

```

UK_USER_EMAIL(email)
IDX_USER_ROLE(role)
IDX_USER_STATUS(status)

```

2. INSURED

보험계약상 피보험자.

사용자 자신뿐 아니라 향후 가족계약 등을 고려하여 **User** 와 분리한다.

컬럼	타입	Key
insured_id	UUID	PK
owner_user_id	UUID	FK
name	VARCHAR(100)	
birth_date	DATE	
gender	VARCHAR(10)	
identity_token	VARCHAR(255)	
relationship_type	VARCHAR(30)	
created_at	TIMESTAMP	

identity_token 은 주민등록번호 평문을 의미하지 않는다.

MVP에서는 주민등록번호 저장을 최소화하고 필요한 경우 별도 보호영역을 검토한다.

3. CONSENT

사용자의 동의 이력을 저장한다.

컬럼	타입
consent_id	UUID
user_id	UUID
consent_type	VARCHAR(50)
consent_version	VARCHAR(30)
agreed	BOOLEAN
agreed_at	TIMESTAMP
revoked_at	TIMESTAMP
source_ip	VARCHAR(45)

예:

```
SERVICE_TERMS  
PRIVACY  
SENSITIVE_INFORMATION  
AI_PROCESSING
```

VII. 보험회사·상품 Domain

4. INSURANCE_COMPANY

컬럼	타입
company_id	UUID
company_code	VARCHAR(30)
company_name	VARCHAR(150)
company_type	VARCHAR(30)
status	VARCHAR(20)
created_at	TIMESTAMP
updated_at	TIMESTAMP

5. INSURANCE_PRODUCT

보험상품의 논리적 Master.

컬럼	타입
product_id	UUID
company_id	UUID FK
product_code	VARCHAR(100)
product_name	VARCHAR(255)
insurance_type	VARCHAR(50)
status	VARCHAR(20)
created_at	TIMESTAMP

예:

```
insurance_type
```

```
THIRD_PARTY
```

```
LIFE
```

```
INDEMNITY
```

```
AUTO
```

```
LIABILITY
```

```
WORKERS_COMP
```

6. PRODUCT_VERSION

보험상품 판매시기별 Version.

컬럼	타입
product_version_id	UUID
product_id	UUID FK
version_name	VARCHAR(100)
sale_start_date	DATE
sale_end_date	DATE
effective_from	DATE
effective_to	DATE
status	VARCHAR(20)

컬럼	타입
created_at	TIMESTAMP

Ⅷ. 보험계약 Domain

7. INSURANCE_CONTRACT

특정 사용자가 실제 가입한 보험계약.

컬럼	타입	Key
contract_id	UUID	PK
user_id	UUID	FK
insured_id	UUID	FK
product_version_id	UUID	FK
policy_version_id	UUID	FK Nullable
policy_number	VARCHAR(100)	
contract_date	DATE	
coverage_start_date	DATE	
coverage_end_date	DATE	
contract_status	VARCHAR(20)	
registration_method	VARCHAR(30)	
source_document_id	UUID	FK Nullable
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

registration_method

MANUAL
OCR
ADMIN
API

8. CONTRACT_COVERAGE

사용자가 실제 가입한 담보.

컬럼	타입
contract_coverage_id	UUID
contract_id	UUID FK
coverage_id	UUID FK
coverage_name_snapshot	VARCHAR(255)
insured_amount	DECIMAL(18,0)
coverage_start_date	DATE
coverage_end_date	DATE
payment_limit	DECIMAL(18,0)
payment_count_limit	INT
status	VARCHAR(20)
created_at	TIMESTAMP

중요

`coverage_name_snapshot` 을 저장한다.

향후 표준 Coverage 이름이 변경되더라도 가입 당시 명칭을 보존하기 위함이다.

IX. 약관 Domain

9. POLICY

약관 논리 Master.

컬럼	타입
policy_id	UUID
product_version_id	UUID FK
policy_name	VARCHAR(255)
policy_type	VARCHAR(30)
status	VARCHAR(20)
created_at	TIMESTAMP

policy_type

GENERAL
SPECIAL
RIDER

10. POLICY_VERSION

약관 Version의 핵심 테이블.

컬럼	타입
policy_version_id	UUID
policy_id	UUID FK
version_code	VARCHAR(100)
effective_from	DATE
effective_to	DATE
published_date	DATE
previous_version_id	UUID FK Nullable
file_hash	VARCHAR(128)
source_file_uri	TEXT
status	VARCHAR(20)
approved_by	UUID FK
approved_at	TIMESTAMP
created_at	TIMESTAMP

상태

DRAFT
PROCESSING
REVIEW_REQUIRED
APPROVED
ACTIVE
DEPRECATED

11. POLICY_CLAUSE

약관 조항.

컬럼	타입
clause_id	UUID
policy_version_id	UUID FK
article_number	VARCHAR(50)
article_title	VARCHAR(255)
paragraph_number	VARCHAR(50)
item_number	VARCHAR(50)
clause_text	TEXT
page_number	INT
source_bbox	JSON
text_hash	VARCHAR(128)
created_at	TIMESTAMP

12. POLICY_APPENDIX

별표·장해분류표·수술분류표 등을 관리한다.

컬럼	타입
appendix_id	UUID
policy_version_id	UUID
appendix_type	VARCHAR(50)
appendix_name	VARCHAR(255)
page_start	INT
page_end	INT
structured_data	JSON
created_at	TIMESTAMP

appendix_type

DISABILITY_TABLE
SURGERY_TABLE
DISEASE_TABLE

PAYMENT_TABLE
OTHER

X. Coverage 및 Rule Domain

13. COVERAGE

보험담보 표준 Master.

컬럼	타입
coverage_id	UUID
standard_code	VARCHAR(100)
coverage_name	VARCHAR(255)
coverage_category	VARCHAR(100)
insurance_type	VARCHAR(50)
description	TEXT
status	VARCHAR(20)

예:

AMI_DIAGNOSIS
CANCER_DIAGNOSIS
DISEASE_SURGERY
INJURY_SURGERY
HOSPITAL_DAILY
FRACTURE_DIAGNOSIS

14. COVERAGE_ALIAS

보험회사마다 담보 명칭이 다를 수 있으므로 Alias를 관리한다.

컬럼	타입
alias_id	UUID
coverage_id	UUID
company_id	UUID Nullable

컬럼	타입
alias_name	VARCHAR(255)
normalized_name	VARCHAR(255)

예:

Coverage:

급성심근경색 진단비

Alias:

급성심근경색증 진단보험금

AMI 진단비

특정심장질환진단비

단, 실제 보장범위가 다른 담보를 단순 Alias로 통합해서는 안 된다.

15. BENEFIT_RULE

Rule의 논리 Master.

컬럼	타입
rule_id	UUID
coverage_id	UUID
policy_version_id	UUID
rule_name	VARCHAR(255)
rule_type	VARCHAR(30)
status	VARCHAR(20)
created_at	TIMESTAMP

rule_type

ELIGIBILITY
EXCLUSION
REDUCTION
CALCULATION
LIMIT

16. RULE_VERSION

실제 실행되는 Rule Version.

컬럼	타입
rule_version_id	UUID
rule_id	UUID
version_no	INT
rule_definition	JSON
effective_from	DATE
effective_to	DATE
status	VARCHAR(20)
source_type	VARCHAR(20)
created_by	UUID
approved_by	UUID
created_at	TIMESTAMP
approved_at	TIMESTAMP

source_type

MANUAL
AI_EXTRACTED
MIGRATED

17. RULE_CONDITION

Rule 조건을 별도 구조화할 경우 사용한다.

컬럼	타입
condition_id	UUID
rule_version_id	UUID
sequence_no	INT
fact_path	VARCHAR(255)
operator	VARCHAR(50)

컬럼	타입
comparison_value	JSON
logical_operator	VARCHAR(10)

예:

```
diagnosis.code
IN
["I21.0", "I21.1", "I21.2", "I21.3", "I21.4", "I21.9"]
```

18. BENEFIT_RULE_CLAUSE

Rule과 약관 근거 조항의 N:M 관계.

컬럼	타입
rule_id	UUID
clause_id	UUID
relation_type	VARCHAR(30)

복합 PK

```
(rule_id, clause_id)
```

XI. Claim Domain

19. CLAIM

보험금 분석 Case의 중심 Entity.

컬럼	타입
claim_id	UUID
user_id	UUID
insured_id	UUID
contract_id	UUID

컬럼	타입
claim_number	VARCHAR(100)
claim_type	VARCHAR(30)
status	VARCHAR(30)
title	VARCHAR(255)
created_at	TIMESTAMP
updated_at	TIMESTAMP
completed_at	TIMESTAMP

상태

DRAFT
 DOCUMENT_REQUIRED
 DOCUMENT_PROCESSING
 USER_VERIFICATION
 ASSESSING
 MANUAL_REVIEW
 COMPLETED
 FAILED
 CANCELLED
 CLOSED

20. ACCIDENT

질병·상해 사고정보.

컬럼	타입
accident_id	UUID
claim_id	UUID
accident_type	VARCHAR(30)
accident_date	DATE
diagnosis_date	DATE
onset_date	DATE
description	TEXT
location	VARCHAR(255)
created_at	TIMESTAMP

accident_type

DISEASE
INJURY
OTHER

XII. 문서 Domain

21. MEDICAL_DOCUMENT

보험금 분석에 사용되는 증거문서.

보험증권도 확장성을 위해 `document_category` 로 관리할 수 있다.

컬럼	타입
document_id	UUID
claim_id	UUID Nullable
user_id	UUID
document_type	VARCHAR(50)
original_filename	VARCHAR(255)
storage_uri	TEXT
file_hash	VARCHAR(128)
mime_type	VARCHAR(100)
file_size	BIGINT
page_count	INT
malware_scan_status	VARCHAR(20)
processing_status	VARCHAR(30)
uploaded_at	TIMESTAMP

document_type

DIAGNOSIS_CERTIFICATE
SURGERY_CERTIFICATE
HOSPITALIZATION_CERTIFICATE
MEDICAL_RECEIPT
MEDICAL_DETAIL

INSURANCE_POLICY
ACCIDENT_REPORT
OTHER

22. OCR_RESULT

문서 OCR 실행결과.

한 문서를 여러 OCR Model로 재처리할 가능성을 고려하여 1:N으로 구성한다.

컬럼	타입
ocr_result_id	UUID
document_id	UUID
provider	VARCHAR(50)
model_version	VARCHAR(100)
raw_text	TEXT
raw_result	JSON
confidence	DECIMAL(5,4)
processing_started_at	TIMESTAMP
processing_completed_at	TIMESTAMP
status	VARCHAR(20)

XIII. Fact Domain

23. EXTRACTED_FACT

AI/OCR이 추출한 사실 후보.

컬럼	타입
extracted_fact_id	UUID
claim_id	UUID
ocr_result_id	UUID
document_id	UUID
fact_type	VARCHAR(50)

컬럼	타입
fact_value	JSON
normalized_value	JSON
confidence	DECIMAL(5,4)
page_number	INT
source_bbox	JSON
extractor_model	VARCHAR(100)
created_at	TIMESTAMP

fact_type 예시

```

DIAGNOSIS_NAME
DIAGNOSIS_CODE
DIAGNOSIS_DATE
SURGERY_NAME
SURGERY_DATE
HOSPITAL_ADMISSION_DATE
HOSPITAL_DISCHARGE_DATE
ACCIDENT_DATE
MEDICAL_FACILITY

```

24. VERIFIED_FACT

보험금 계산에 실제 사용되는 검증 Fact.

컬럼	타입
verified_fact_id	UUID
claim_id	UUID
extracted_fact_id	UUID Nullable
fact_type	VARCHAR(50)
verified_value	JSON
verification_status	VARCHAR(30)
verified_by	UUID
verified_at	TIMESTAMP
modification_reason	VARCHAR(255)
source_document_id	UUID

컬럼	타입
source_page	INT
source_bbox	JSON

verification_status

AI_ONLY
 USER_CONFIRMED
 USER_MODIFIED
 EXPERT_CONFIRMED
 EXPERT_MODIFIED
 REJECTED

핵심 Rule

Rule Engine 입력 허용:

USER_CONFIRMED
 USER_MODIFIED
 EXPERT_CONFIRMED
 EXPERT_MODIFIED

중요 Fact에서는 **AI_ONLY** 를 자동계산에 사용하지 않는 정책을 적용할 수 있다.

XIV. 의료정보 세부 Entity

MVP에서는 VerifiedFact JSON으로도 구현할 수 있지만 향후 의료정보 검색·분석을 위해 별도 Entity를 두는 것을 권장한다.

25. DIAGNOSIS

컬럼	타입
diagnosis_id	UUID
claim_id	UUID
verified_fact_id	UUID
diagnosis_name	VARCHAR(255)
diagnosis_code	VARCHAR(30)

컬럼	타입
code_system	VARCHAR(20)
diagnosis_date	DATE
is_primary	BOOLEAN

26. SURGERY

컬럼	타입
surgery_id	UUID
claim_id	UUID
surgery_name	VARCHAR(255)
normalized_surgery_name	VARCHAR(255)
surgery_date	DATE
body_part	VARCHAR(100)
verified_fact_id	UUID

27. HOSPITALIZATION

컬럼	타입
hospitalization_id	UUID
claim_id	UUID
admission_date	DATE
discharge_date	DATE
calculated_days	INT
hospital_name	VARCHAR(255)
verified_fact_id	UUID

XV. 보험금 판단 Domain

28. COVERAGE_ASSESSMENT

Claim 내의 각 가입담보별 지급판단.

예:

Claim 1개

암진단비 평가
질병수술비 평가
입원일당 평가

각각 하나의 `CoverageAssessment` 가 된다.

컬럼	타입
assessment_id	UUID
claim_id	UUID
contract_coverage_id	UUID
policy_version_id	UUID
rule_version_id	UUID Nullable
assessment_status	VARCHAR(30)
match_score	DECIMAL(5,4)
eligibility_result	VARCHAR(30)
exclusion_result	VARCHAR(30)
reduction_result	VARCHAR(30)
reason_summary	TEXT
assessed_at	TIMESTAMP

eligibility_result

PAYABLE
LIKELY_PAYABLE
ADDITIONAL_INFO_REQUIRED
MANUAL_REVIEW
LIKELY_NOT_PAYABLE
POSSIBLE_EXCLUSION
NOT_PAYABLE
UNDETERMINED

29. ASSESSMENT_RULE_RESULT

여러 Rule이 적용되는 경우 개별 Rule 평가결과를 저장한다.

컬럼	타입
result_id	UUID
assessment_id	UUID
rule_version_id	UUID
rule_type	VARCHAR(30)
result	VARCHAR(30)
input_snapshot	JSON
execution_trace	JSON
executed_at	TIMESTAMP

XVI. 보험금 계산 Domain

30. BENEFIT_CALCULATION

실제 보험금 계산결과.

컬럼	타입
calculation_id	UUID
assessment_id	UUID
claim_id	UUID
contract_coverage_id	UUID
rule_version_id	UUID
currency	CHAR(3)
insured_amount_snapshot	DECIMAL(18,0)
payment_rate_snapshot	DECIMAL(10,6)
calculation_formula	TEXT
calculation_input	JSON
gross_amount	DECIMAL(18,0)
deduction_amount	DECIMAL(18,0)

컬럼	타입
final_amount	DECIMAL(18,0)
calculation_status	VARCHAR(30)
calculated_at	TIMESTAMP

중요

다음은 반드시 Snapshot으로 저장한다.

```
insured_amount_snapshot
payment_rate_snapshot
calculation_formula
calculation_input
```

31. CLAIM_CALCULATION_SUMMARY

성능상 필요하면 Claim 총액을 별도로 관리한다.

컬럼	타입
summary_id	UUID
claim_id	UUID
total_expected_amount	DECIMAL(18,0)
payable_count	INT
review_count	INT
calculated_at	TIMESTAMP

단, 원본 계산결과는 항상 **BenefitCalculation** 이다.

XVII. Evidence Chain

32. EVIDENCE

판단과 계산의 근거를 관리한다.

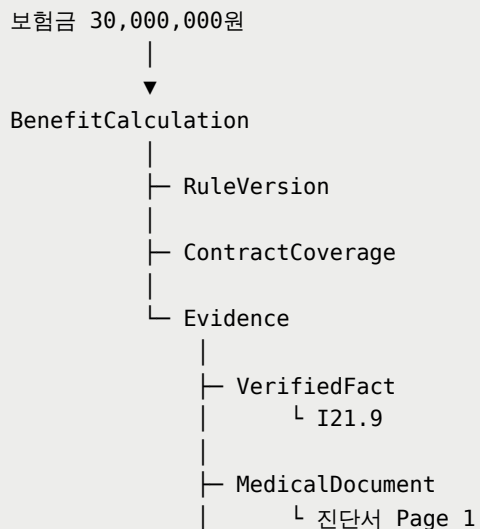
컬럼	타입
evidence_id	UUID

컬럼	타입
calculation_id	UUID Nullable
assessment_id	UUID Nullable
evidence_type	VARCHAR(30)
verified_fact_id	UUID Nullable
clause_id	UUID Nullable
document_id	UUID Nullable
rule_version_id	UUID Nullable
page_number	INT Nullable
source_bbox	JSON Nullable
description	TEXT
created_at	TIMESTAMP

evidence_type

MEDICAL_FACT
 POLICY_CLAUSE
 CONTRACT
 RULE
 DOCUMENT
 CALCULATION

XVIII. Evidence 관계 예시



XIX. Human-in-the-loop Domain

33. REVIEW

컬럼	타입
review_id	UUID
claim_id	UUID
assessment_id	UUID Nullable
reviewer_user_id	UUID
review_type	VARCHAR(30)
review_status	VARCHAR(30)
reason	TEXT
opinion	TEXT
previous_result	JSON
final_result	JSON
requested_at	TIMESTAMP
started_at	TIMESTAMP
completed_at	TIMESTAMP

review_status

REQUESTED
ASSIGNED
IN_PROGRESS
ADDITIONAL_DOCUMENT_REQUIRED
APPROVED
MODIFIED
UNDETERMINED
COMPLETED

34. REVIEW_ASSIGNMENT

여러 전문가에게 Queue를 배정할 가능성을 고려하면 별도 Entity 사용 가능.

컬럼	타입
assignment_id	UUID
review_id	UUID
reviewer_user_id	UUID
assigned_by	UUID
assigned_at	TIMESTAMP
accepted_at	TIMESTAMP

MVP에서 전문가 1명 직접배정이면 **Review**에 통합 가능하다.

XX. Audit Domain

35. AUDIT_LOG

컬럼	타입
audit_id	UUID
actor_user_id	UUID Nullable
event_type	VARCHAR(50)
object_type	VARCHAR(50)
object_id	UUID Nullable
claim_id	UUID Nullable
request_id	VARCHAR(100)
source_ip	VARCHAR(45)
before_value	JSON
after_value	JSON
result	VARCHAR(20)
created_at	TIMESTAMP

Audit 대상 이벤트

```
LOGIN
LOGIN_FAIL
DOCUMENT_UPLOAD
DOCUMENT_VIEW
DOCUMENT_DOWNLOAD
OCR_EXECUTE
FACT_MODIFY
CONTRACT_CREATE
CONTRACT_MODIFY
RULE_CREATE
RULE_MODIFY
RULE_TEST
RULE_APPROVE
RULE_DEPLOY
CALCULATION_EXECUTE
CALCULATION_RECALCULATE
REVIEW_OVERRIDE
PERMISSION_CHANGE
```

XXI. Rule Version과 계산결과 관계

가장 중요한 관계 중 하나다.

```
BenefitRule
|
├─ RuleVersion 1
├─ RuleVersion 2
└─ RuleVersion 3
```

2026년 1월 Case:

```
Calculation
→ RuleVersion 1
```

2026년 8월 Rule 개정:

```
RuleVersion 3
```

이때 과거 계산은:

```
Calculation
```

→ RuleVersion 1

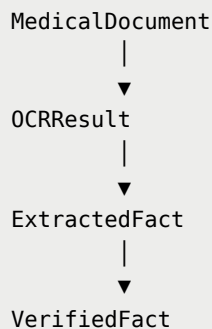
을 계속 유지한다.

XXII. 약관 Version 선택 관계



`Claim`에서는 계산 당시 선택된 `PolicyVersion`을 `CoverageAssessment`에 저장한다.

XXIII. Document → Fact 구조



예:

Document
진단서.pdf
↓
OCRResult
"급성심근경색증 I21.9"

```
↓  
  
ExtractedFact  
  
fact_type:  
DIAGNOSIS_CODE  
  
value:  
I21.9  
  
confidence:  
0.96  
  
↓  
  
VerifiedFact  
  
value:  
I21.9  
  
status:  
USER_CONFIRMED
```

XXIV. 중요 데이터 무결성 Constraint

1. InsuranceContract

```
contract_date <= coverage_start_date  
coverage_start_date < coverage_end_date
```

2. PolicyVersion

동일 Policy 내에서 ACTIVE Version의 적용기간이 의도치 않게 중복되지 않도록 검증한다.

3. ContractCoverage

```
insured_amount >= 0
```

4. Hospitalization


```
admission_date <= discharge_date
```

5. BenefitCalculation

```
final_amount >= 0
```

6. RuleVersion

ACTIVE 전환 조건:

```
approved_by IS NOT NULL  
approved_at IS NOT NULL  
status = ACTIVE
```

XXV. Soft Delete 정책

다음 정보는 물리 삭제보다 Soft Delete 또는 비활성화가 적합하다.

- InsuranceCompany
- InsuranceProduct
- Coverage
- Policy
- Rule

과거 보험금 산정 근거가 사라져서는 안 되기 때문이다.

XXVI. 물리 삭제가 필요한 정보

개인정보 보관기간 종료 등 법적 파기 요구가 있는 사용자 데이터는 별도 삭제 또는 익명화 정책을 적용한다.

예:

```
User
Insured
MedicalDocument
VerifiedFact
```

단, Audit 및 법적 보존의무와 충돌하는 경우 별도의 Data Retention 설계가 필요하다.

XXVII. 개인정보와 Domain DB 분리 권장

권장 구조:

```
[Identity / PII]
User
Insured
Consent

      | reference

[Insurance Domain]
Contract
Claim
Coverage
Calculation

      | reference

[Document Storage]
Encrypted Medical Documents
```

보험분석 Database에 불필요한 직접식별정보를 반복 저장하지 않는다.

XXVIII. 파일 Storage와 DB 분리

진단서 PDF/Image 자체를 RDB Blob으로 저장하지 않는 방식을 기본으로 권장한다.

```
Object Storage
  |
  } encrypted document
  |
  ▼
MedicalDocument
  storage_uri
```

DB에는 Metadata와 URI를 저장한다.

XXIX. 중요 Index 설계

Claim 검색

```
IDX_CLAIM_USER_STATUS  
(user_id, status)
```

사용자 계약

```
IDX_CONTRACT_USER  
(user_id)
```

```
IDX_CONTRACT_INSURED  
(insured_id)
```

Policy Version

```
IDX_POLICY_VERSION_PERIOD  
(policy_id, effective_from, effective_to)
```

Rule

```
IDX_RULE_COVERAGE  
(coverage_id)
```

```
IDX_RULE_VERSION_STATUS  
(rule_id, status)
```

Fact

```
IDX_FACT_CLAIM_TYPE  
(claim_id, fact_type)
```

계산

```
IDX_CALC_CLAIM  
(claim_id)
```

```
IDX_CALC_ASSESSMENT  
(assessment_id)
```

Audit

```
IDX_AUDIT_OBJECT  
(object_type, object_id)
```

```
IDX_AUDIT_ACTOR_TIME  
(actor_user_id, created_at)
```

XXX. ID 전략

모든 주요 Entity는 UUID 사용을 권장한다.

예:

```
claim_id  
contract_id  
document_id  
rule_id  
calculation_id
```

외부 사용자에게 노출하는 번호는 별도로 생성한다.

예:

```
claim_id  
550e8400-e29b-41d4-a716-446655440000  
  
claim_number  
CLM-20260818-000001
```

XXXI. 금액 타입

보험금은 **FLOAT** 또는 **DOUBLE** 사용을 피한다.

권장:

```
DECIMAL(18,0)
```

대한민국 원 단위 계산 중심인 MVP 기준.

금액 외 지급률:

```
DECIMAL(10,6)
```

XXXII. 날짜와 시간

업무상 날짜:

```
DATE
```

예:

- 사고일
- 진단일
- 계약일

시스템 이벤트:

```
TIMESTAMP WITH TIME ZONE
```

권장.

예:

- created_at
- approved_at
- calculated_at

XXXIII. JSON 사용 범위

JSON 사용에 적합:

- OCR Raw Result
- AI Raw Result
- Rule DSL
- Bounding Box
- Calculation Input Snapshot
- Execution Trace

관계검색이 빈번한 중요 데이터는 일반 Column으로 분리한다.

잘못된 구조

```
{
  "contract": {
    "product": "...",
    "coverage": [...],
    "claim": {...}
  }
}
```

전체 보험업무를 하나의 JSON에 저장하지 않는다.

XXXIV. MVP 핵심 ERD만 축약

1차 구현에 반드시 필요한 테이블은 다음과 같다.

```
USER
INSURED

INSURANCE_COMPANY
INSURANCE_PRODUCT
PRODUCT_VERSION

INSURANCE_CONTRACT
CONTRACT_COVERAGE

POLICY
POLICY_VERSION
```

POLICY_CLAUSE

COVERAGE

BENEFIT_RULE
RULE_VERSION

CLAIM
ACCIDENT

MEDICAL_DOCUMENT
OCR_RESULT
EXTRACTED_FACT
VERIFIED_FACT

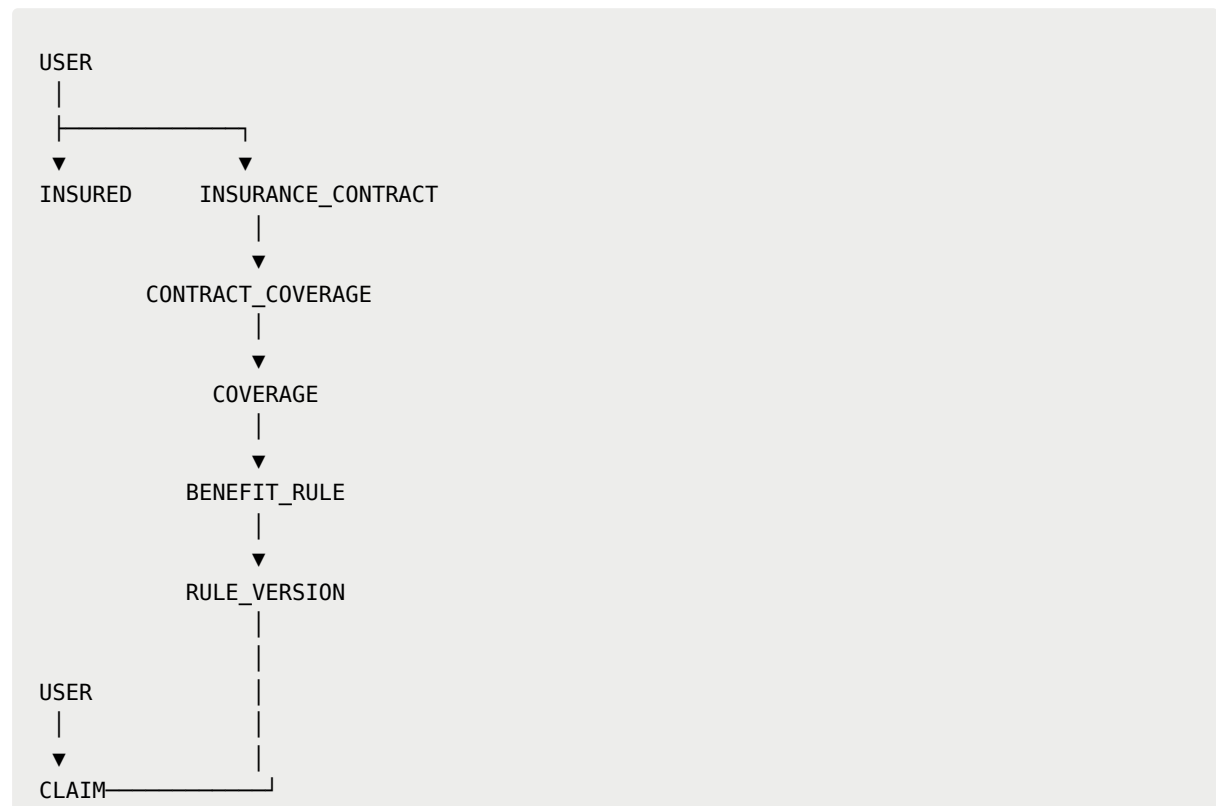
COVERAGE_ASSESSMENT
ASSESSMENT_RULE_RESULT

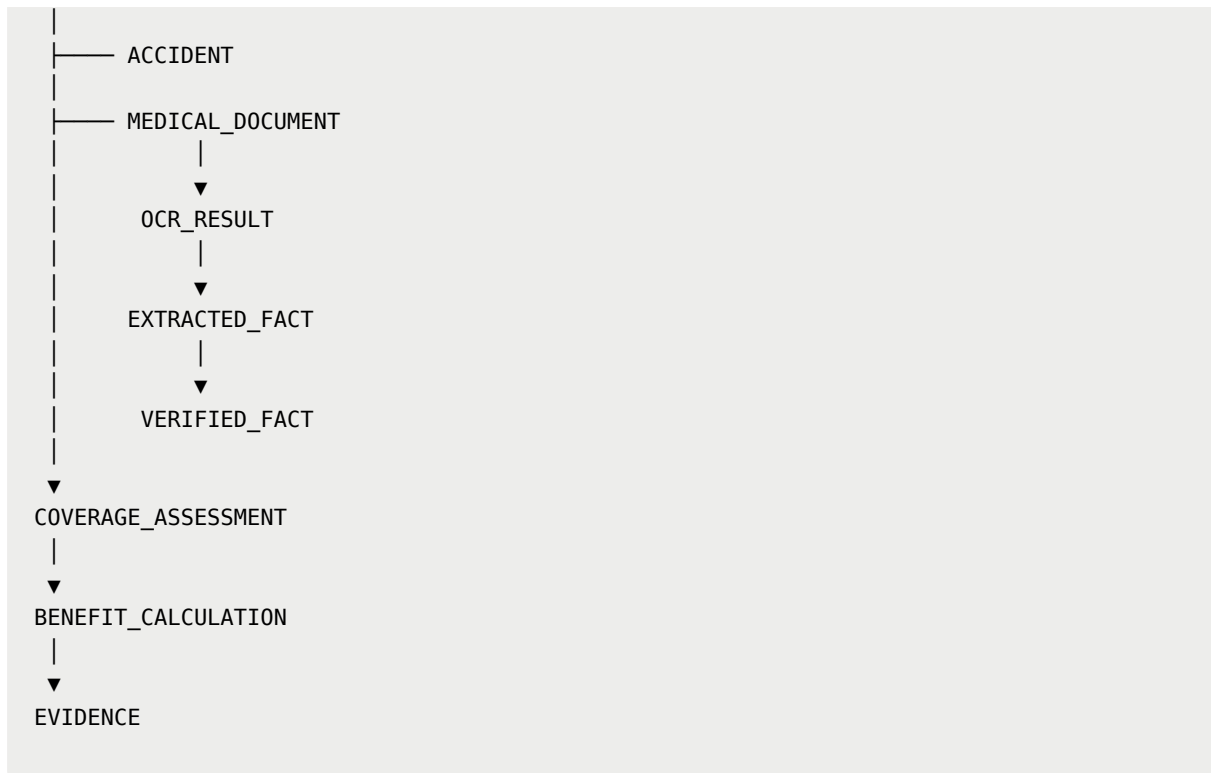
BENEFIT_CALCULATION
EVIDENCE

REVIEW
AUDIT_LOG

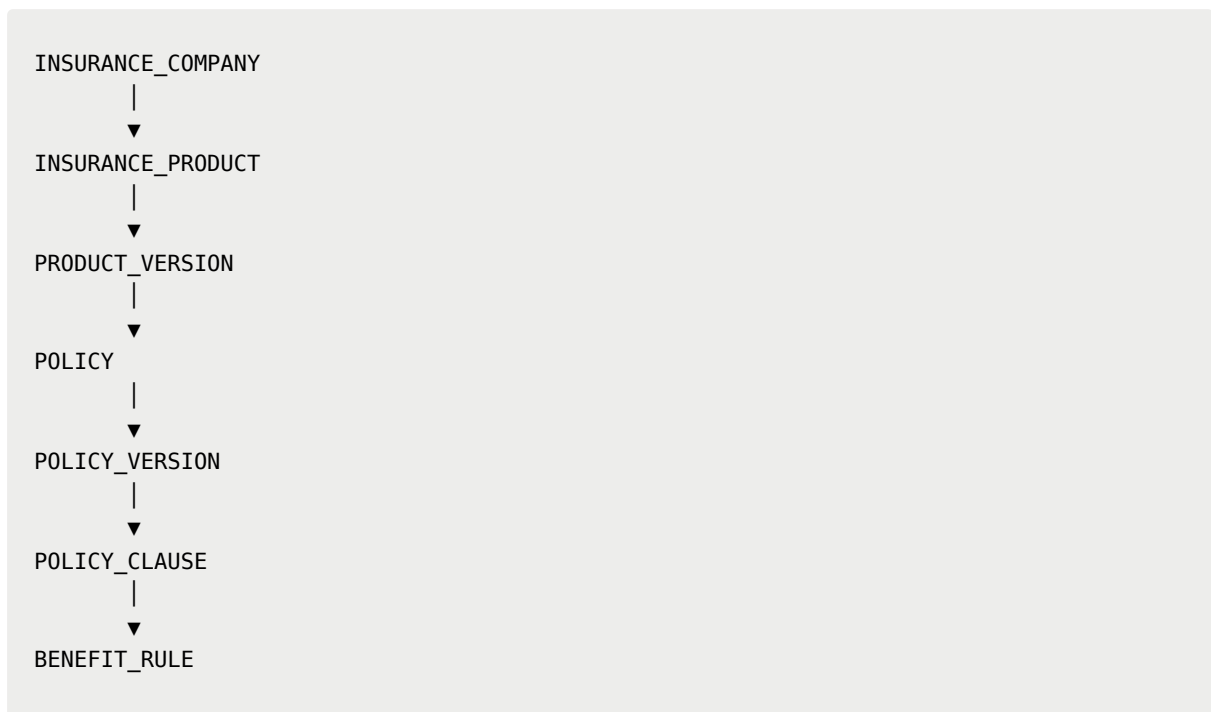
약 **25개 핵심 테이블**로 MVP Domain을 구성한다.

XXXV. MVP ERD 축약도





약관:



XXXVI. 보험금 계산 시 Database Query Flow

예:

급성심근경색증 진단비

```
1. Claim 조회
  ↓
2. Contract 조회
  ↓
3. ContractCoverage 조회
  ↓
4. VerifiedFact 조회

diagnosis.code = I21.9
diagnosis.date = 2026-07-10

  ↓
5. PolicyVersion 확인
  ↓
6. Active RuleVersion 확인
  ↓
7. Rule Condition 평가
  ↓
8. CoverageAssessment 저장
  ↓
9. BenefitCalculation 생성
  ↓
10. Evidence 생성
```

XXXVII. 보험금 계산 Snapshot 예시

```
{
  "claim_id": "CLM-001",
  "contract_coverage_id": "CC-001",

  "facts": {
    "diagnosis_code": "I21.9",
    "diagnosis_date": "2026-07-10"
  },

  "contract": {
    "insured_amount": 30000000
  },

  "policy": {
    "policy_version": "2025.04"
```

```

    },

    "rule": {
      "rule_version": 3,
      "payment_rate": 1.0
    },

    "calculation": {
      "formula": "insured_amount * payment_rate",
      "result": 30000000
    }
  }
}

```

XXXVIII. 재계산 설계

VerifiedFact가 변경되면 기존 계산결과를 덮어쓰지 않는 방식을 권장한다.

예:

Calculation V1
30,000,000원

↓

진단코드 수정

↓

Calculation V2
0원

다음 필드를 추가할 수 있다.

```

calculation_version
previous_calculation_id
is_current

```

XXXIX. BenefitCalculation 개선안

최종 권장 구조:

컬럼	설명
calculation_id	PK
assessment_id	지급판단
calculation_version	계산 Version
previous_calculation_id	이전 계산
is_current	현재 계산
rule_version_id	적용 Rule
input_snapshot	입력
formula	계산식
gross_amount	계산 전
deduction_amount	공제
final_amount	최종액
calculated_at	계산일

XL. Rule 배포 이력

추후 운영환경을 고려하면 다음 Entity 추가를 권장한다.

RULE_DEPLOYMENT

```
deployment_id  
rule_version_id  
environment  
deployed_by  
deployed_at  
rollback_of  
status
```

환경:

```
DEV  
TEST  
STAGING  
PRODUCTION
```

MVP 초기에는 AuditLog로 대체 가능하다.

XLI. AI Model 추적

보험금 오류 분석을 위해 OCR 및 AI 모델 Version을 기록한다.

추가 후보 Entity:

AI_MODEL_EXECUTION

```
execution_id  
claim_id  
document_id  
provider  
model_name  
model_version  
prompt_version  
input_hash  
output_hash  
started_at  
completed_at  
status
```

AI 모델 변경 전후 결과 비교가 가능해진다.

MVP에서는 `OCR_RESULT` 와 `EXTRACTED_FACT` 에 Model Version을 기록하는 것으로 단순화 가능하다.

XLII. 데이터 정합성 핵심 Rule

보험금 계산이 실행되기 위한 조건:

```
InsuranceContract 존재  
AND  
ContractCoverage 존재  
AND  
Claim 존재  
AND  
PolicyVersion 확정  
AND  
VerifiedFact 존재  
AND  
RuleVersion 확정
```

중요 Fact가 미검증인 경우:

Calculation 실행 금지
OR
MANUAL_REVIEW 전환

XLIII. ERD와 화면 Mapping

화면	주요 Entity
SCR-001	User
SCR-003	Consent
SCR-010	InsuranceContract, Claim
SCR-020	InsuranceContract
SCR-021	InsuranceContract, ContractCoverage
SCR-022	MedicalDocument, OCRResult
SCR-030	Claim
SCR-031	Accident
SCR-040	MedicalDocument
SCR-041	OCRResult
SCR-042	ExtractedFact, VerifiedFact
SCR-050	CoverageAssessment
SCR-060	BenefitCalculation
SCR-061	BenefitCalculation, Evidence
SCR-062	PolicyVersion, PolicyClause
SCR-063	CoverageAssessment
SCR-070	Review
ADM-SCR-030	Policy
ADM-SCR-031	PolicyVersion
ADM-SCR-040	PolicyClause
ADM-SCR-050	Coverage
ADM-SCR-060	BenefitRule, RuleVersion
ADM-SCR-061	RuleVersion, AssessmentRuleResult

화면	주요 Entity
ADM-SCR-062	RuleVersion
ADM-SCR-080	AuditLog

XLIV. ERD와 API Mapping

예:

```

POST /contracts
→ INSURANCE_CONTRACT

POST /contracts/{id}/coverages
→ CONTRACT_COVERAGE

POST /claims
→ CLAIM + ACCIDENT

POST /claims/{id}/documents
→ MEDICAL_DOCUMENT

POST /documents/{id}/ocr
→ OCR_RESULT

GET /claims/{id}/facts
→ EXTRACTED_FACT + VERIFIED_FACT

PATCH /facts/{id}
→ VERIFIED_FACT

POST /claims/{id}/assess
→ COVERAGE_ASSESSMENT

POST /claims/{id}/calculate
→ BENEFIT_CALCULATION

GET /calculations/{id}/evidence
→ EVIDENCE

```

XLV. MVP 구현 순서

Database 구현은 다음 순서가 적합하다.

1. User / Insured
- ↓
2. InsuranceCompany

↓
3. InsuranceProduct / ProductVersion
↓
4. Policy / PolicyVersion
↓
5. PolicyClause
↓
6. Coverage
↓
7. BenefitRule / RuleVersion
↓
8. InsuranceContract
↓
9. ContractCoverage
↓
10. Claim / Accident
↓
11. MedicalDocument
↓
12. OCRResult
↓
13. ExtractedFact
↓
14. VerifiedFact
↓
15. CoverageAssessment
↓
16. BenefitCalculation
↓
17. Evidence
↓
18. Review
↓
19. AuditLog

XLVI. 개발 시 특히 주의할 ERD 10개 포인트

1. InsuranceProduct 와 InsuranceContract 를 합치지 않는다.
2. Coverage 와 ContractCoverage 를 합치지 않는다.
3. Policy 와 PolicyVersion 을 합치지 않는다.
4. Rule 은 Version을 반드시 관리한다.
5. OCR 결과를 보험금 계산에 직접 사용하지 않는다.
6. ExtractedFact 와 VerifiedFact 를 구분한다.
7. 보험금 계산결과는 Snapshot을 저장한다.

8. 과거 계산을 신규 Rule로 자동 갱신하지 않는다.
9. Evidence를 계산결과와 직접 연결한다.
10. Rule·Fact·계산 변경은 AuditLog에 남긴다.

XLVII. 최종 핵심 데이터 흐름



XLVIII. 최종 ERD 설계문장

보험금 계산의 중심 Entity는 Claim이지만, 계산의 신뢰성은 ContractCoverage·PolicyVersion·RuleVersion·VerifiedFact가 동시에 결합되어 확보된다.

따라서 시스템의 핵심 관계는 다음과 같이 정의한다.

```
Claim
+
ContractCoverage
+
PolicyVersion
+
RuleVersion
+
VerifiedFact
=
CoverageAssessment
+
BenefitCalculation
+
Evidence
```

이 구조를 유지하면 향후 보험종목이 추가되더라도 사용자·계약·문서·증거 구조를 다시 만들 필요 없이 **Coverage와 Rule Engine을 확장하는 방식**으로 대응할 수 있다.

XLIX. 다음 단계

ERD 이후에는 ERD의 Entity와 실제 Component를 연결하는 **시스템 아키텍처 설계서**를 작성한다.

권장 순서:

```
ERD
↓
시스템 아키텍처
↓
보험약관 데이터 모델 상세설계
↓
Policy / Rule Engine 상세설계
↓
OCR·AI Pipeline 상세설계
↓
API 명세
↓
보안 아키텍처
↓
테스트 시나리오
```

↓
개발 Backlog
↓
AGENTS.md
↓
Codex 구현 Prompt

다음 산출물인 시스템 아키텍처에서는 최소 다음 Component를 정의한다.

Frontend
API Gateway
Authentication Service
Contract Service
Claim Service
Document Service
OCR Service
AI Extraction Service
Policy Service
Coverage Matching Service
Policy Version Resolver
Eligibility Engine
Calculation Engine
Evidence Service
Review Service
Audit Service
RDBMS
Object Storage
Vector/Search Engine

그리고 어떤 **Service**가 위 ERD의 어떤 **Entity**를 소유하는지까지 명확하게 정의한다.